Python for Social Scientists

A Practical Introduction for Researchers

EC34: Data Science for Economists

Swarthmore College

Spring 2026

Why Python?

The Reproducibility Crisis

The Problem

- You finish a paper. A referee asks: "What happens if you drop observations before 1990?"
- You open your Excel file. You can't remember which cells you edited, which columns you deleted, or in what order
- *60% of economics papers* cannot be reproduced, even with the original data (Chang & Li, 2022)

The Solution: Code

- A script is a **complete record** of every step from raw data to final result
- Anyone can re-run it and get the *exact same output*
- Change one assumption? Edit one line, re-run everything
- Your future self will thank you

If your analysis cannot be reproduced, it cannot be verified.

Why Python?

- **Free and open-source** — no license fees, no institutional restrictions, runs anywhere
- **The most widely used language in data science** — more tutorials, answers, and community support than any alternative
- **Not just statistics** — the same language can scrape websites, call APIs, process text, analyze satellite images, and run machine learning models
- **Readable** — Python reads almost like English, making it easier to learn and share with collaborators
- **Interoperable** — you can call Stata and R *from within* Python, so nothing is lost

```python
# This is valid Python — readable, concise, expressive
df = pd.read_csv("survey_data.csv")
avg_income = df.groupby("region")["income"].mean()
```

Python vs. Other Tools

	Stata	R	Excel	Python
Cost	Expensive	Free	Paid	Free
Econometrics	Excellent	Excellent	Limited	Very Good
Machine Learning	Limited	Good	None	Excellent
Data Wrangling	Good	Excellent	Poor	Excellent
Visualization	Good	Excellent	Good	Excellent
Web Scraping	None	Moderate	None	Excellent
Text / NLP / AI	None	Good	None	Excellent
Scalability	Limited	Moderate	Very Limited	Excellent

Python is not the best at any single task — it is the best *general-purpose* tool for modern social science.

The Python Ecosystem for Social Science

There is a mature Python library for nearly everything a social scientist needs:

Data & Econometrics

- **pandas** — data manipulation
- **statsmodels** — regression, MLE
- **pyfixest** — panel fixed effects
- **rdrobust** — regression discontinuity

Visualization & Mapping

- **matplotlib / seaborn** — static plots
- **plotly** — interactive charts
- **geopandas / folium** — maps
- **geemap** — satellite imagery

ML, Text & AI

- **scikit-learn** — machine learning
- **doubleml / econml** — causal ML
- **openai** — LLM APIs
- **pytorch** — deep learning

You don't need to learn all of these. Most projects use only 3-4.

Jupyter Notebooks: Your Lab Notebook

Jupyter notebooks let you combine **writing**, **code**, and **results** in a single interactive document — like a lab notebook for data analysis.

- Write explanations in plain English alongside your code
- See tables, plots, and maps appear *immediately* below each code cell
- Share the notebook on GitHub — anyone can read your analysis and reproduce it
- Export to PDF for journal submission

```python
# Load the data
df = pd.read_csv("survey_data.csv")

# Average income by region
df.groupby("region")["income"].mean()
```

A formatted table appears here instantly.

```python
# Visualize the distribution
df["income"].plot(kind="hist")
```

A histogram appears here instantly.

Working with Data

DataFrames: Programmable Spreadsheets

The core idea: a **DataFrame** is a table — rows are observations, columns are variables — but every operation is a *function call* rather than a mouse click.

What you can do:

- Load data from CSV, Excel, Stata, SQL databases, APIs, and the web
- Inspect your data: see the first few rows, summary statistics, missing values
- Filter, sort, and subset with one line
- Handles *millions* of rows without crashing (try that in Excel)

```python
df = pd.read_csv("data/census.csv")

# Summary statistics in one line
df.describe()

# Filter to just one state
ca = df.query("state ==
'California'")
```

Cleaning Messy Data

Real-world data is *always* messy — missing values, inconsistent formatting, duplicates, wrong types. Python gives you systematic tools to handle all of this reproducibly.

- **Missing values** — detect, drop, or impute them programmatically
- **Duplicates** — identify and remove in one line
- **Type conversion** — turn strings into dates, fix numeric columns stored as text
- **Standardization** — clean up inconsistent names, capitalization, whitespace

In most projects, *80% of the work* is cleaning data. Python makes this reproducible rather than ad-hoc.

Merging, Grouping & Reshaping

Merging Datasets

Combine a household survey with census tract data by matching on FIPS codes — like Stata's `merge` or Excel's VLOOKUP, but for millions of rows.

Grouping & Aggregation

"What is the average income by state and year?" becomes one line of code. Compute means, medians, standard deviations, or custom functions across any grouping.

Reshaping

Convert between **long** format (one row per observation-year) and **wide** format (one row per observation, years as columns) — essential for panel data.

Method Chaining

String operations together in readable pipelines:

```
result = (df
    .query("year >= 2000")
    .groupby("region")["income"]
    .mean())
```

Working with Text Data

Social scientists increasingly work with **unstructured text** — survey responses, legislative transcripts, historical documents, news articles, social media.

What Python can do with text:

- Extract structured data (dates, dollar amounts, names) from free text
- Search for keywords and patterns across thousands of documents
- Clean and standardize messy text fields
- Classify documents by topic or sentiment

```python
# Extract all 4-digit years
df["year"] = df["text"].str.extract(
    r"(\d{4})"
)

# Flag documents about inflation
df["about_inflation"] = (
    df["text"].str.contains(
        "inflation|CPI|prices"
    )
)
```

Panel Data

Panel data — repeated observations of the same units over time — is the backbone of empirical social science. Python handles it natively.

- **Index by unit and time** — track households, firms, or countries across years
- **Within-group operations** — demean variables, compute lags, calculate growth rates, all by group
- **Reshape freely** — switch between long and wide formats for different analyses
- **Merge in time-varying covariates** — match external data by unit-year

```python
# Create a lagged income variable for each household
df["lag_income"] = df.groupby("household_id")["income"].shift(1)

# Demean income within each household (for fixed effects intuition)
df["income_demean"] = df.groupby("household_id")["income"].transform("mean")
```

Visualization

Publication-Quality Plots

Python can produce **journal-ready figures** — scatter plots, line charts, histograms, bar charts, box plots, and more — with full control over every visual element.

- **Seaborn** generates beautiful statistical graphics in a single function call — regression lines with confidence bands, distribution plots, correlation heatmaps
- **Matplotlib** gives fine-grained control when you need to customize every axis label, annotation, and color
- Export directly to PDF, PNG, or SVG for LaTeX

```python
# Scatter plot with regression line
# and 95% confidence band
sns.lmplot(
    x="education", y="income",
    data=df, hue="gender", ci=95
)
plt.savefig("figure1.pdf", dpi=300)
```

One function call produces a publication-ready figure.

Interactive Visualization

Static plots are great for papers, but **interactive charts** are powerful for exploration and presentations.

With Plotly, you can:

- **Hover** over data points to see details (country name, exact value)
- **Zoom** into regions of interest
- **Animate** over time — watch trends unfold year by year
- **Filter** by category interactively
- Embed in Jupyter notebooks, websites, or presentations

The "Hans Rosling" Plot

```
px.scatter(
    gapminder,
    x="gdpPercap", y="lifeExp",
    size="pop", color="continent",
    animation_frame="year",
    hover_name="country"
)
```

GDP vs. life expectancy for every country, animated from 1952 to 2007, with population as bubble size.

Maps & Geospatial Analysis

Choropleth Maps

Color states, counties, or countries by any variable — poverty rates, election margins, treatment intensity.

- Load shapefiles (geographic boundaries)
- Merge with your economic data by FIPS code or country name
- Generate interactive maps: zoom, click, hover for details
- No GIS software needed — everything in Python

Satellite Imagery

Access **Google Earth Engine** from Python to work with:

- **Nighttime lights** — a proxy for economic activity where official GDP data is unreliable
- **Vegetation indices** — measure agricultural productivity
- **Urban expansion** — track development over decades
- **Deforestation** — monitor environmental policy impacts

All from a Jupyter notebook.

Econometrics in Python

Regression Analysis

Python supports the full toolkit of regression analysis that social scientists rely on, with syntax that feels familiar if you know Stata or R.

What's available:

- **OLS** with R-style formulas
- Heteroskedasticity-robust standard errors (HC0–HC3)
- Cluster-robust standard errors
- Weighted least squares
- Instrumental variables (2SLS)
- Full regression tables with coefficients, SEs, t-stats, R^2

```python
import statsmodels.formula.api as smf

# Familiar formula syntax
model = smf.ols(
    "wage ~ education + experience",
    data=df
).fit(cov_type="HC3")

model.summary()
```

$$\text{wage}_i = \beta_0 + \beta_1 \text{educ}_i + \beta_2 \text{exper}_i + u_i$$

Panel Data & Fixed Effects

Fixed effects models are the workhorse of applied microeconomics — control for all time-invariant unobserved heterogeneity.

$$y_{it} = \beta x_{it} + \alpha_i + \delta_t + \varepsilon_{it}$$

- **Entity fixed effects** (α_i) — absorb unobserved unit characteristics
- **Time fixed effects** (δ_t) — absorb common shocks
- **Clustered SEs** — account for within-group correlation

Python's `pyfixest` is the equivalent of Stata's `reghdfe`:

```
import pyfixest as pf

# Two-way fixed effects
mod = pf.feols(
    "wage ~ education"
    " | worker + year",
    data=panel
)
mod.summary()
```

Handles high-dimensional fixed effects efficiently — thousands of entities and time periods.

Causal Inference: The Identification Problem

The central challenge of empirical social science: **correlation is not causation.**

- Does education *cause* higher wages, or do high-ability people both earn more and stay in school longer?
- Does a policy *cause* the outcome, or did treated units differ from control units all along?

Python has mature tools for every major identification strategy.

The Causal Toolkit

Strategy	Python Package
Fixed Effects	`pyfixest`
Difference-in-Differences	`pyfixest`
Instrumental Variables	`linearmodels`
Regression Discontinuity	`rdrobust`
Matching	`causalml`
Synthetic Control	`SparseSC`

Difference-in-Differences

The idea: compare the change in outcomes for a treated group to the change for a control group. The key assumption is that both groups would have followed **parallel trends** without treatment.

$$= \beta_0 + \beta_1 \text{Treat}_i + \beta_2 \text{Post}_t + \beta_3 (\text{Treat}_i \times \text{Post}_t) + \varepsilon_{it}$$

β_3 is the **treatment effect** — the causal impact of the policy.

Applications: minimum wage laws, healthcare expansions, environmental regulations, school funding changes.

Modern advances (available in Python):

- **Event studies** — estimate dynamic effects before and after treatment
- **Staggered adoption** — units adopt treatment at different times; new estimators (Callaway & Sant'Anna) avoid bias from "forbidden comparisons"
- **Synthetic DiD** — combine matching with DiD

Regression Discontinuity

The idea: when treatment is assigned by a cutoff in a running variable (test scores, vote margins, income thresholds), units *just above* and *just below* the cutoff are effectively randomized.

$$\tau = \lim_{x \to c^+} E[Y \mid X = x] - \lim_{x \to c^-} E[Y \mid X = x]$$

Applications:

- Class size effects (Maimonides' Rule)
- Close elections and incumbency advantage
- Eligibility cutoffs for social programs
- GPA thresholds for academic probation

The `rdrobust` package handles everything:

- **Data-driven bandwidth selection** — automatically chooses how close to the cutoff to focus
- **Local polynomial estimation** — flexible functional form near the cutoff
- **Robust confidence intervals**
- **Built-in visualization** — see the discontinuity

The exact same package used by researchers in Stata and R.

Simulation & Bootstrap

Monte Carlo Simulation

Generate data where you **know the truth**, then test whether your estimator recovers it.

- Is my estimator unbiased?
- How large a sample do I need to detect an effect? (**power analysis**)
- What happens when my assumptions are violated?
- Essential for evaluating *any* new method

Bootstrap Inference

When analytical standard errors are unreliable:

- **Few clusters** — wild bootstrap gives better inference than cluster-robust SEs
- **Complex estimators** — bootstrap gives standard errors even when formulas don't exist
- **Heteroskedasticity** — robust to distributional assumptions

Python makes it trivial to run 10,000 bootstrap replications.

Maximum Likelihood & Beyond

Maximum Likelihood Estimation

Many models in social science — logit, probit, Tobit, selection models — are estimated via MLE.

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \sum_{i=1}^{n} \log f(y_i \mid x_i, \theta)$$

In Python, you can use **built-in models** (logit, probit) or **define your own likelihood function** and let the optimizer find the parameters.

Generalized Method of Moments

- Estimate models defined by **moment conditions** rather than a full likelihood
- IV estimation as a special case
- Overidentification tests

Python gives you the building blocks to implement *any* estimator — standard or custom — in a reproducible, testable way.

Machine Learning & AI

What Machine Learning Offers Social Scientists

Prediction

- Which students are at risk of dropping out?
- What will unemployment be next quarter?
- Where should we target inspections?

ML excels at finding complex patterns in data, even when you don't know the functional form in advance.

Strengthening Causal Inference

- **Variable selection** — which controls matter among hundreds of candidates?
- **Nuisance estimation** — flexibly estimate confounding without imposing parametric assumptions
- **Heterogeneity** — who benefits most from a policy?

ML is not a substitute for identification — it is a *tool that strengthens it.*

The ML Workflow

The key discipline of machine learning: **never evaluate a model on the data it was trained on.**

1. **Split** the data into training and test sets
2. **Train** the model on the training set
3. **Evaluate** on the held-out test set
4. **Cross-validate** to tune hyperparameters

This prevents **overfitting** — the model learning noise rather than signal.

```
from sklearn.model_selection import (
    train_test_split
)
from sklearn.linear_model import
LassoCV

X_train, X_test, y_train, y_test = (
    train_test_split(X, y)
)

# LASSO: automatic variable selection
model = LassoCV(cv=5).fit(
    X_train, y_train
)
```

Causal Machine Learning: Double ML

The problem: you want the causal effect of D on Y, but you have hundreds of potential confounders X. Which controls should you include?

Double ML (Chernozhukov et al., 2018):

1. Use ML to predict Y from X (flexibly)
2. Use ML to predict D from X (flexibly)
3. Regress the *residuals* of Y on the *residuals* of D
4. The coefficient is the **causal effect** of D

No need to choose a parametric specification for the confounders. ML handles the "nuisance" so you can focus on the causal question.

```python
from doubleml import (
    DoubleMLData, DoubleMLPLR
)

dml = DoubleMLPLR(
    data,
    ml_l=LassoCV(),  # predict Y
    ml_m=LassoCV(),  # predict D
    n_folds=5
)
dml.fit()
print(dml.summary)
```

Who Benefits? Heterogeneous Treatment Effects

A policy might help some people and hurt others. Traditional methods estimate **one average effect**. Causal forests let you ask: **for whom is the effect largest?**

Causal Forests (Athey & Wager, 2019):

- Estimate a *personalized* treatment effect $\tau(x)$ for each individual based on their characteristics
- Data-driven — the algorithm discovers which subgroups matter
- **SHAP values** explain *why* the model predicts larger effects for certain groups

Applications:

- Which patients benefit most from a drug?
- Which job seekers benefit most from a training program?
- Where should a government target a subsidy?
- How should an NGO allocate limited resources?

These questions were very hard to answer before causal ML.

Text as Data

Text is everywhere in social science — but it used to be nearly impossible to analyze at scale. Python changes that.

Applications:

- Measure **political polarization** from congressional speeches
- Classify **sentiment** in consumer reviews or earnings calls
- Extract **policy positions** from party manifestos
- Analyze **media framing** of events across outlets

Tools available:

- **Pattern matching** — find dates, dollar amounts, names in documents
- **Keyword classification** — flag documents by topic
- **Sentiment analysis** — positive vs. negative language
- **Large Language Models** — use GPT-4 or Claude to classify, summarize, or extract information from text at scale, via Python API calls

Computer Vision & Historical Data

Digitizing the Past

Many historical sources exist only as scanned images — census forms, newspaper advertisements, handwritten ledgers, maps.

Python can:

- **OCR** — extract text from scanned documents
- **Segment images** — identify tables, figures, handwriting
- **Classify** — sort documents by type automatically
- Build datasets that would take *years* to create manually

Satellite & Aerial Imagery

- Measure **land use change** over decades
- Detect **informal settlements** in developing countries
- Track **infrastructure development** (roads, buildings)
- Classify **crop types** for agricultural economics
- All using image classification models trained in Python

Economists are increasingly using images as data — and Python is the primary tool for this work.

Deep Learning: When and Why

When Deep Learning

- **Unstructured data** — images, audio, text
- **Very large datasets** — hundreds of thousands of observations
- **Prediction** is the primary goal
- Feature engineering would be prohibitively complex

Python (via **PyTorch**) is the dominant language for deep learning — the same tool used by OpenAI, Google DeepMind, and Meta AI.

When Econometrics

- **Causal questions** require identification strategies, not just pattern recognition
- **Small/medium datasets** — where overfitting is a real concern
- **Interpretability** is essential for policy implications
- **Theory** guides the model specification

The two approaches are *complements*, not substitutes. The best empirical work increasingly uses both.

Reproducible Workflow

Organizing a Research Project

A well-organized project ensures that anyone (including your future self) can understand and reproduce your work.

Key Principles

- **Separate raw data from processed data** — never modify original files
- **Number your scripts** — make execution order obvious
- **One script per task** — cleaning, analysis, figures
- **Relative paths** — the project runs on any computer

```
my-project/
├── data/
│   ├── raw/        # Never modify
│   └── clean/      # Script-
generated
├── code/
│   ├── 01-clean.py
│   ├── 02-analyze.py
│   └── 03-figures.py
├── output/
│   ├── tables/
│   └── figures/
└── paper.qmd
```

Version Control with Git

Git tracks every change you make to your code — who changed what, when, and why. It is the standard for collaborative research.

- **Never lose work** — every version is saved; revert mistakes instantly
- **Experiment freely** — create branches to try different specifications without affecting your main analysis
- **Collaborate** — multiple coauthors can work on the same project without emailing files back and forth
- **GitHub** hosts your project online — free backup, sharing, and peer review
- **Code review** — coauthors can comment on specific lines before changes are merged
- **Transparency** — share your full analysis history alongside your paper
- Many journals now *require* code repositories

Git is not just for programmers — it is for anyone who writes code for research.

Dynamic Documents with Quarto

The dream: change your data or model, and every table, figure, and number in your paper updates automatically.

Quarto makes this real:

- Write your paper in **Markdown** with embedded Python code
- Tables and figures are *generated by code*, not copy-pasted
- One command produces PDF, HTML, Word, or slides
- BibTeX citations, cross-references, LaTeX math — all supported
- No more "the numbers in Table 3 don't match the text"

```
---
title: "The Effect of X on Y"
format: pdf
---

## Data

```{python}
df = pd.read_csv("data/clean.csv")
df.describe()
```

Results

Table 1 shows that...
```

Interoperability

You Don't Have to Abandon Stata

If you have years of Stata code, you don't need to rewrite it. **pystata** lets you run Stata commands *from within* a Python notebook.

Use cases:

- Keep your existing `.do` files and call them from Python
- Use Stata for estimation, Python for data wrangling and visualization
- Transfer data seamlessly between the two environments
- Gradually learn Python without giving up your current workflow

```python
from pystata import stata

# Run Stata commands directly
stata.run("sysuse auto, clear")
stata.run("reg price mpg weight")

# Send your Python data to Stata
stata.pdataframe_to_data(df)

# Get Stata results back in Python
df = stata.pdataframe_from_data()

# Run an existing .do file
stata.run("do my_analysis.do")
```

You Don't Have to Abandon R Either

R has excellent packages for certain tasks — `ggplot2` for visualization, `sf` for spatial data, Bayesian methods with `Stan`. With **rpy2**, you can use them all from Python.

Use cases:

- Access the **tidyverse** when it fits your workflow
- Use specialized R packages that have no Python equivalent
- Run R code in the *same notebook* as your Python analysis
- Share data between R and Python seamlessly

```python
import rpy2.robjects as ro
from rpy2.robjects import pandas2ri
pandas2ri.activate()

# Pass your DataFrame to R
ro.globalenv["df"] = df

# Use R packages
ro.r("""
library(ggplot2)
ggplot(df, aes(x=income, y=health))
  + geom_point()
  + geom_smooth(method="lm")
""")
```

Conclusion

1. **Python is a complement**, not a replacement — use it alongside Stata and R
2. **Reproducibility** comes from scripted, version-controlled workflows
3. **Data wrangling** at scale — clean, merge, reshape millions of rows
4. **Visualization** from static journal figures to interactive animated maps
5. **Econometrics** in Python is mature — OLS, fixed effects, DiD, RDD, MLE
6. **Machine learning** strengthens causal inference — Double ML, causal forests
7. **New frontiers** — text as data, satellite imagery, LLMs, computer vision
8. **Interoperability** — call Stata and R from within Python

Python is not about replacing your toolkit — it is about expanding it.